

Prompt de Construção — Sistema de Dados Reais Tastytrade

Especificação completa para começar do zero, projeto novo e separado do RADAR-TASYTRADE

Preparado para Duda

08 de setembro de 2026

Como usar este documento

Cole o conteúdo da seção “PROMPT” abaixo, na íntegra, para iniciar a construção — seja nesta sessão, seja em uma sessão nova, seja para outro desenvolvedor. Ele foi escrito para ser autossuficiente: não depende de nenhum contexto de conversa anterior.

As decisões de escopo abaixo refletem o que você confirmou: lista real dos Top 10 por volume + campo de busca para o restante do SP500. Duas decisões que você não respondeu explicitamente foram assumidas com o padrão mais seguro (declarado abaixo) — revise antes de começar, porque mudam a arquitetura.

Decisões confirmadas por você nesta rodada: 1. **Ambiente: local agora, Vercel depois.** Primeiro validar 100% local (Streamlit), migrar para nuvem só depois de confirmado. Por causa disso, o cache de token já nasce neste projeto no padrão memória-primeiro / disco-como-contingência-best-effort (nunca dependência rígida de escrita em disco) — é o mesmo ajuste que corrigiu esse risco no RADAR, e evita ter que reescrever essa parte quando for migrar. 2. **Credenciais: mesma conta Tastytrade, arquivo .env próprio e separado.** “Reaproveitar” aqui significa uma coisa específica e nada mágica: você (ou um comando local seu) copia os mesmos três valores que já estão em `RADAR-TASYTRADE/.env.local` (`CLIENT_ID`, `CLIENT_SECRET`, `REFRESH_TOKEN`) para um novo arquivo `.env` deste projeto novo. Ninguém além de você manuseia o valor real dessas credenciais — elas nunca passaram por esta conversa, nunca foram lidas por mim, e não há nenhuma transferência automática entre projetos. É a mesma conta, duplicada manualmente em dois arquivos de configuração locais.

PROMPT

Construa um sistema simples, do zero, em Python + Streamlit, rodando localmente, com UMA regra inegociável acima de qualquer outra consideração de escopo, performance ou conveniência:

NUNCA fabricar, simular, modelar, interpolar ou estimar nenhum dado de mercado. Todo número exibido vem de uma resposta real da API da Tastytrade, nesta mesma execução. Se um dado não estiver disponível na resposta real, o campo mostra "indisponível" — nunca um valor calculado, nunca um fallback numérico, nunca um "valor típico". Nenhuma outra fonte de dado além da Tastytrade é usada em nenhuma hipótese, mesmo que a Tastytrade não tenha o dado (a resposta correta nesse caso é "indisponível", não buscar em outro provedor).

Escopo funcional

1. TELA PRINCIPAL: lista real dos Top 10 ativos do S&P 500 por volume de negociação do dia, ordenada de forma decrescente. Para cada um, mostrar: símbolo, nome, último preço, variação % do dia, volume real do dia.
2. CAMPO DE BUSCA: campo de texto onde o usuário digita qualquer ticker do S&P 500 (fora do Top 10) e recebe os mesmos dados abaixo.
3. PARA O ATIVO SELECIONADO (seja do Top 10, seja da busca), mostrar:
 - a. Cotação real: último preço, bid, ask, variação %, volume do dia.
 - b. IV Rank e IV Percentile reais (não confundir um com o outro — são métricas diferentes, ambas existem na API de market-metrics da

Tastytrade).

- c. Cadeia de opções real (todos os vencimentos disponíveis, selecionável pelo usuário) com, para cada strike/vencimento escolhido:
 - Delta, Gamma, Theta, Vega, Rho reais (não calculados por Black-Scholes local – vêm da API real)
 - IV (volatilidade implícita) real por contrato
 - Open Interest real, separado por CALL e por PUT
 - Bid/Ask/Mid reais do contrato

Fonte de dados – só Tastytrade, endpoints confirmados

Autenticação: OAuth2 com refresh_token, endpoint POST /oauth/token, exatamente como já implementado e testado no projeto RADAR-TASYTRADE local (reaproveitar a lógica de troca de refresh_token por access_token – mas escrito do zero neste projeto novo, não copiado do RADAR). As credenciais (CLIENT_ID, CLIENT_SECRET, REFRESH_TOKEN) vêm de um arquivo .env PRÓPRIO deste projeto, preenchido manualmente por você com os mesmos valores que já estão no .env.local do RADAR – nenhuma automação copia isso, e nenhum agente/IA tem ou precisa ter acesso ao valor real dessas credenciais em nenhum momento.

Cache de token: desde o primeiro commit, seguir o padrão memória-primeiro (variável em processo) com disco como contingência melhor-esforço dentro de try/except (nunca lançar erro se a escrita falhar). Mesmo rodando local agora – onde escrita em disco funciona sem restrição – esse padrão evita retrabalho quando migrar para Vercel depois, onde o filesystem é efêmero e majoritariamente somente-leitura fora de /tmp.

Fontes REST confirmadas (via a OpenAPI oficial da Tastytrade, <https://developer.tastytrade.com/openapi/market-data.json>):

- Cotação de ações em lote: GET /market-data/by-type?equity=<símbolos separados por vírgula> – retorna bid/ask/last/volume real por símbolo. USE ESTE ENDPOINT para montar o ranking real de Top 10 por volume: buscar em lote os símbolos do universo do S&P 500 (lista de tickers apenas – não preços – pode vir de uma lista estática de componentes do índice, já que isso não é "dado de mercado fabricado", é só a composição do índice) e ordenar localmente pelo campo de volume real retornado. ANTES DE IMPLEMENTAR: confirmar empiricamente o limite de símbolos por chamada desse endpoint (pode não aceitar ~500 de uma vez – testar e, se precisar, paginar em lotes).
- Estrutura da cadeia de opções (vencimentos e strikes disponíveis): GET /option-chains/{symbol}/nested
- Cotação real por contrato de opção: GET /market-data/by-type?equity-option=<símbolos OCC separados por vírgula>
- IV Rank / IV Percentile / IV30: GET /market-metrics?symbols=<símbolos> (campos confirmados: ivRank, ivPercentile, iv30)

Fonte de streaming (obrigatória para Greeks e Open Interest – a API REST da Tastytrade NÃO expõe delta/gamma/theta/vega/rho nem open interest):

- Protocolo DXLink via WebSocket. Token e URL vêm de GET /api-quote-tokens (campos data.token e data['dxlink-url']).
- Sequência: SETUP (canal 0) → aguardar AUTH_STATE UNAUTHORIZED → enviar AUTH com o token → aguardar AUTHORIZED → CHANNEL_REQUEST (canal 1, service FEED, parameters {contract: "AUTO"}) → FEED_SUBSCRIPTION (adicionar símbolos com type "Greeks" para delta/gamma/theta/vega/rho/IV, e type "Summary" para Open Interest – CONFIRMAR empiricamente qual tipo de evento carrega dayOpenInterest antes de assumir; no dashboard Python de referência que você validou, essa mecânica já funciona – usar exatamente o mesmo padrão de mensagens).
- Símbolos usados no streaming são os "streamer symbols" (formato .TICKER+YYMMDD+C/P+STRIKE), diferentes do símbolo OCC usado no REST. A cadeia retornada por /option-chains/{symbol}/nested já traz os dois formatos por strike – usar o campo correto para cada chamada.

Regras de honestidade de dado (não negociáveis)

- Se a chamada de IV Rank/Percentile falhar ou não retornar o símbolo,

- mostrar "indisponível" nesses campos – nunca herdar de outro ativo, nunca usar 50% como "neutro" default.
- Se o streaming DXLink não responder dentro de um tempo limite razoável (ex.: 10-12 segundos), mostrar "indisponível" nos gregos/IV/OI daquele contrato – nunca preencher com Black-Scholes local.
- Se o Open Interest não vier separado por CALL/PUT na resposta real, mostrar "indisponível" nesse campo – nunca estimar proporção.
- Nunca usar uma lista estática de preços/volumes/IV como fallback. A única lista estática permitida no projeto é a composição do índice S&P 500 (só os tickers – não valores de mercado).

Entrega

- App Streamlit local, um único comando para rodar (``streamlit run app.py`` ou equivalente).
- Projeto em pasta nova e separada de qualquer código do RADAR-TASYTRADE – nenhum arquivo copiado de lá, só as credenciais (mesma conta Tastytrade) configuradas via variável de ambiente / `.env` próprio deste projeto novo.
- Critério de aceite: abrir o app, ver o Top 10 real batendo com a plataforma oficial da Tastytrade no mesmo instante (mesmo critério visual que você já usou para pegar o achado da NVDA/BAC – comparação lado a lado com a tela real da corretora, não "parece razoável").

O que fica de fora deste escopo (e por quê)

Fora do escopo agora	Motivo
Recomendação de estratégia de opções (Iron Condor, Calendar etc.)	Foi a causa da complexidade que gerou os problemas anteriores — este sistema novo entrega só o dado real bruto primeiro; estratégia é uma camada posterior, só depois do dado bruto estar validado
Deploy em nuvem / Vercel	Confirmado: local primeiro. Migrar para nuvem depois de validado — o cache de token já é desenhado desde o início para não precisar de retrabalho nessa migração (ver seção de Autenticação acima), mas o timeout de função serverless (achado do RADAR: streaming DXLink pode levar ~10s, plano Hobby da Vercel corta em 10s) ainda vai precisar ser reavaliado nesse momento, não antes
Histórico de candlestick (OHLCV)	Não fazia parte do que você pediu agora (cotação + cadeia + gregas + IV Rank/Percentile + OI). Se quiser depois, precisa de uma frente própria via streaming DXLink (evento Candle)
Screenener com todos os +30 filtros do RADAR antigo	Fora do pedido — este é um sistema simples e novo, não uma migração do RADAR